

CANLIFAL — FLUTTER ↔ BACKEND ENTEGRASYON SÖZLEŞMESİ

Sürüm: 1.0 · Tarih: 2026-08-27 · Kapsam: Flutter mobil istemcisinin CanlıFal backend'i ile konuşurken uyması gereken tüm kurallar.

Bu belge **sözleşmedir**. Buradaki alan adları, olay adları ve zarf yapıları geriye dönük uyumlu tutulur; kırıcı değişiklik yapılmadan önce bu dosya güncellenir.

1. BASE URL

Ortam	Base URL
Üretim	<code>https://canlifal.com</code>
Önizleme (geliştirme)	konuşma bazlı önizleme adresi

İki eşdeğer API öneki desteklenir:

```
https://canlifal.com/api/<yol>
https://canlifal.com/api/v1/<yol> ← önerilen (mobil için sürüm sabitlemesi)
```

`/api/v1/*` istekleri sunucu tarafında `/api/*` yoluna yeniden yazılır. **İkisi bit düzeyinde aynı yanıtı döner**. Flutter tarafında `/api/v1` kullanın; ileride v2 çıkarsa v1 çalışmaya devam eder.

Zorunlu istek başlıkları:

```
Accept: application/json
Content-Type: application/json      (gövdeli isteklerde)
Authorization: Bearer <accessToken> (kimlik gerektiren uçlarda)
```

2. AUTHENTICATION

Mobil istemci **JWT Bearer** kullanır. Web tarafındaki oturum çerezi mobilde kullanılmaz. Tüm korumalı uçlar çift kimlik doğrular: önce `Authorization` başlığı, yoksa web oturumu.

Giriş uçları

Uç	Amaç
POST /api/v1/auth/mobile-register	E-posta + parola ile kayıt
POST /api/v1/auth/mobile-login	E-posta + parola ile giriş
POST /api/v1/auth/mobile-google	Google ID token ile giriş
POST /api/v1/auth/mobile-apple	Apple identity token ile giriş
POST /api/v1/auth/mobile-tiktok	TikTok OAuth ile giriş
POST /api/v1/auth/mobile-refresh	Access token yenileme
POST /api/v1/auth/logout	Oturumu kapat
POST /api/v1/auth/forgot-password	Parola sıfırlama e-postası
POST /api/v1/auth/reset-password	Token ile parola sıfırlama
POST /api/v1/auth/change-password	Oturum içi parola değişimi
POST /api/v1/auth/verify-device	Cihaz doğrulama
POST /api/v1/auth/reclaim-device	Cihazı yeni hesaba taşıma

Giriş isteği/yanıtı

POST /api/v1/auth/mobile-login

Content-Type: application/json

```
{ "email": "user@example.com", "password": "*****" }
```

```
{
  "success": true,
  "user": { "id": "clx...", "email": "user@example.com", "name": "Ayşe", "role": "user", "image": null },
  "accessToken": "eyJhbGciOi...",
  "refreshToken": "eyJhbGciOi..."
}
```

⚠ DİKKAT: accessToken ve refreshToken yanıtın **kökündedir**, data içinde değil. Bu uç tarihsel sebeplerle standart zarfı kullanmaz ve geriye dönük uyum için böyle kalacaktır.

3. TOKEN LIFECYCLE

Token	Ömür	Nerede saklanır
accessToken	7 gün	flutter_secure_storage
refreshToken	30 gün	flutter_secure_storage

JWT yükü: { userId, email, role, type: 'access'|'refresh', iat, exp }.

Yenileme akışı

1. Her istekte Authorization: Bearer <accessToken> gönder.
2. 401 UNAUTHORIZED alırsan:
POST /api/v1/auth/mobile-refresh { "refreshToken": "..."}
→ yeni { accessToken, refreshToken }
3. Orijinal isteği YENİ access token ile BİR KEZ tekrarla.
4. Yenileme de 401 dönerse → tokenları sil, giriş ekranına yönlendir.

Kurallar:

- Aynı anda yalnızca **tek** yenileme isteği çalışsın (mutex). Paralel 401'ler tek yenilemeyi beklesin.
- Yenileme kuyruğu döngüye girmesin: bir istek en fazla **1 kez** yeniden denenir.
- Access token süresi dolmadan **12 saat** önce arka planda proaktif yenileme önerilir.
- Çıkışta iki token da silinir ve POST /api/v1/auth/logout çağrılır.

4. API ENDPOINTS

Tam liste: backend-docs/ENDPOINTS.md , backend-docs/openapi.json , backend-docs/postman_collection.json .

Mobil için kritik uç grupları:

Grup	Örnek uçlar
Açılış	GET /api/v1/bootstrap , GET /api/v1/config , GET /api/v1/health
Kullanıcı	/api/v1/me , /api/v1/user/* , /api/v1/profile , /api/v1/users/*
Cüzdan	/api/v1/wallet , /api/v1/jeton , /api/v1/credit-packages , /api/v1/payments , /api/v1/withdrawals
Hediye	/api/v1/gifts/catalog , /api/v1/gifts/send , /api/v1/gifts/battles , /api/v1/gifts/missions , /api/v1/gifts/insights , /api/v1/gifts/lucky
Sohbet odası	/api/v1/chat/rooms , /api/v1/chat/rooms/{roomId}/state , .../messages , .../gifts , .../seats , .../presence , .../voice , .../pk , .../stream
Canlı yayın	/api/v1/video-streams , /api/v1/video-streams/{streamId} , .../gifts , .../stream , /api/v1/video-streams/pk
Canlı oda (fal)	/api/v1/live/* , /api/v1/room/{sessionId}/stream , /api/v1/fortune-tellers/*
PK	/api/v1/pk/active , /api/v1/pk/me , /api/v1/pk/{matchId} , /api/v1/pk/{matchId}/stream , /api/v1/pk/leaderboard
Fal	/api/v1/fortunes/* (14 fal türü, SSE akışlı)
Sosyal	/api/v1/social , /api/v1/messages , /api/v1/notifications , /api/v1/stories , /api/v1/short-videos
Gerçek zamanlı	/api/v1/rtc/telemetry , /api/v1/trtc/user-sig , /api/v1/trtc/token
Cihaz	/api/v1/devices/fcm
Yükleme	/api/v1/upload/presigned , /api/v1/upload/get-url , /api/v1/short-videos/upload-url
Derin bağlantı	/api/v1/deeplink/resolve

5. REQUEST EXAMPLES

Cüzdan bakiyesi

```
GET /api/v1/wallet
Authorization: Bearer <accessToken>
```

Hediye gönderme (idempotent)

```
POST /api/v1/gifts/send
Authorization: Bearer <accessToken>
Idempotency-Key: 7f1c3e0a-5b2d-4a9e-8f11-6c0d2a4b8e33
Content-Type: application/json

{ "giftId": "gift rose", "recipientId": "clx...", "quantity": 5, "roomId": "clr..." }
```

Sayfalı liste (imleç modu)

```
GET /api/v1/notifications?paginate=cursor&limit=30
GET /api/v1/notifications?cursor=clx9abc...&limit=30
Authorization: Bearer <accessToken>
```

Oda mesajı gönderme

```
POST /api/v1/chat/rooms/clr123/messages
Authorization: Bearer <accessToken>

{ "content": "Merhaba" }
```

6. RESPONSE EXAMPLES

Standart zarf (yeni uçlar)

Başarı

```
{
  "success": true,
  "data": { "...": "..." },
  "request_id": "req_mtbvnrvq_lkgtki92"
}
```

Sayfalı başarı

```
{
  "success": true,
  "data": [ { "...": "..." } ],
  "meta": { "cursor": "clx9abc...", "hasMore": true, "limit": 30, "total": 142 },
  "request_id": "req_..."
}
```

Hata

```
{
  "success": false,
  "error": { "code": "INSUFFICIENT_BALANCE", "message": "Yetersiz bakiye", "details":
    { "required": 500, "available": 120 } },
  "request_id": "req_..."
}
```

Eski (legacy) format

Uçların bir kısmı hâlâ şu formatı döner ve **bilinçli olarak korunmaktadır**:

```
{ "error": "Recipient is required" }
```

Flutter kuralı — her iki formatı da anlayan tek ayrıştırıcı yazın:

```
ApiResult parse(int status, Map<String, dynamic> b) {
  if (b['success'] == true) return ApiResult.ok(b['data'] ?? b, b['meta']);
  if (b['success'] == false) return ApiResult.err(b['error']['code'], b['error']['message']);
  if (b.containsKey('error')) return ApiResult.err('LEGACY_ERROR', b['error'].toString());
  return status < 400 ? ApiResult.ok(b, null) : ApiResult.err('UNKNOWN', 'Bilinmeyen hata');
}
```

`request_id` alanını hata raporlarınıza ekleyin — sunucu loglarıyla eşleştirmek için tek anahtardır.

7. ERROR CODES

Tam katalog: `lib/ERROR_CODES.md` (111 kod). HTTP durumu ile eşleşme özeti:

HTTP	Anlamı	Tipik kodlar
400	Geçersiz istek	VALIDATION_ERROR , INVALID_ID_INPUT , MISSING_FIELD , INVALID_AMOUNT
401	Kimlik yok/geçersiz	UNAUTHORIZED , TOKEN_EXPIRED , INVALID_TOKEN
403	Yetki yok	FORBIDDEN , BANNED , MUTED , NOT_ROOM_OWNER , KYC_REQUIRED
404	Bulunamadı	NOT_FOUND , ROOM_NOT_FOUND , STREAM_NOT_FOUND , USER_NOT_FOUND
409	Çakışma	DUPLICATE_REQUEST , SEAT_TAKEN , ALREADY_EXISTS , PK_ALREADY_ACTIVE
422	İş kuralı	INSUFFICIENT_BALANCE , LIMIT_EXCEEDED , INVALID_STATE
429	Hız sınırı	RATE_LIMITED
500	Sunucu	INTERNAL_ERROR
503	Servis dışı	SERVICE_UNAVAILABLE

Kural: Flutter tarafında `error.code` üzerinden dallanın, `message` üzerinden **asla**. `message` kullanıcıya gösterilecek Türkçe metindir ve haber verilmeden değişebilir; `code` sözleşmenin parçasıdır.

8. WEBSOCKET URL

Bu platformda WebSocket YOKTUR. Gerçek zamanlı katman **Server-Sent Events (SSE)** üzerindedir. Bu mimari bir karardır; Redis/WebSocket altyapısı kullanılmaz.

Flutter tarafında `web_socket_channel` değil, **HTTP akış istemcisi** kullanın (`http.Client().send(Request(...))` + `Stream<List<int>>` satır ayrıştırma, veya `sse_channel` benzeri bir paket).

Akış	URL
Sohbet/sesli oda	GET /api/v1/chat/rooms/{roomId}/stream
Canlı yayın	GET /api/v1/video-streams/{streamId}/stream
Fal oturumu (oda)	GET /api/v1/room/{sessionId}/stream
Falcı oturum istekleri	GET /api/v1/fortune-tellers/sessions/stream
PK maçı	GET /api/v1/pk/{matchId}/stream
Bildirimler	GET /api/v1/notifications/stream
Fal üretimi (14 tür)	GET /api/v1/fortunes/{tur} — token token akış

SSE çerçeve formatı:

```
data: {"type":"...", ...}

id: 1756318203912

: heartbeat
```

9. WEBSOCKET AUTHENTICATION

SSE bağlantıları da **Authorization: Bearer <accessToken>** başlığı ile kimliklenir. Tarayıcının Event-Source API'si özel başlık desteklemez; Flutter'da bu bir kısıt değildir — ham HTTP isteği açın ve başlığı ekleyin.

```
final req = http.Request('GET', Uri.parse('$base/api/v1/chat/rooms/$roomId/stream'))
  ..headers['Authorization'] = 'Bearer $accessToken'
  ..headers['Accept'] = 'text/event-stream'
  ..headers['Cache-Control'] = 'no-cache';
if (lastEventId != null) req.headers['Last-Event-ID'] = lastEventId;
final res = await http.Client().send(req);
```

Yetki sonuçları:

Durum	Anlam	İstemci davranışı
200	Akış açıldı	Dinle
403	Odadan yasaklısın	Yeniden bağlanma, kullanıcıyı bilgilendir
404	Oda/yayın yok	Yeniden bağlanma, ekrandan çık
401	Token geçersiz	Token yenile, bir kez tekrar dene

Anonim (tokensız) bağlantı **okuma amaçlı izin verilir** — sunucu `userId=anonymous` olarak loglar; kişiye özel olaylar (typing hariç tutma vb.) gelmez.

10. EVENT TYPES

Oda akışı — `/chat/rooms/{roomId}/stream`

type	Ne zaman
<code>connected</code>	Akış açılır açılmaz (ilk çerçeve)
<code>messages</code>	Yeni sohbet mesaj(lar)ı (dizi)
<code>system</code>	Moderasyon: kick, ban, mute, duyuru, sohbet temizleme
<code>gift</code>	Odada hediye gönderildi
<code>pk</code>	PK skoru/durumu değişti
<code>room_event</code>	Sesli oda olayı — alt tür <code>event</code> alanında
<code>presence</code>	Aktif kullanıcı listesi (10 sn'de bir tam liste)
<code>dj</code>	Müzik/DJ durumu değişti
<code>typing</code>	Yazıyor göstergesi

`room_event` alt türleri (`event` alanı)

`user_joined`, `user_left`, `mic_changed`, `seat_changed`, `room_closed`, `owner_changed`, `voice_request`, `hand_raised`, `voice_request_cancelled`, `voice_request_accepted`, `voice_request_rejected`, `voice_request_blocked`, `voice_request_unblocked`, `pk_invite`, `pk_requested`

Diğer akışlar

Akış	type değerleri
/video-streams/{id}/stream	connected , streamMessage , gift , presence , pk
/notifications/stream	connected , notification
/room/{sessionId}/stream	message , session_request
/fortunes/*	ham metin parçaları (token akışı) + bitiş işareti

İleri uyumluluk kuralı: Bilinmeyen bir `type` veya `event` geldiğinde Flutter **sessizce yok saymalıdır**, hata atmamalıdır. Yeni olaylar haber verilmeden eklenebilir.

11. EVENT PAYLOADS

```

// connected
{ "type": "connected", "roomId": "clr123" }

// messages
{ "type": "messages", "messages": [
  { "id": "cm1", "content": "Merhaba", "userId": "u1", "userName": "Ayşe",
    "userImage": null, "createdAt": "2026-08-27T18:00:00.000Z", "roleSymbol": "👑" }
]}

// system
{ "type": "system", "action": "kick", "targetUserId": "u2", "by": "u1", "reason":
"spam" }

// gift
{ "type": "gift", "giftId": "g1", "giftName": "Gül", "giftImage": "https://<cdn>/
gifts/rose.webp",
  "quantity": 5, "totalJeton": 500,
  "senderId": "u1", "senderName": "Ayşe", "recipientId": "u2", "recipientName":
"Mehmet",
  "ts": 1756318203912 }

// pk
{ "type": "pk", "battleId": "pk1", "status": "active",
  "score1": 1200, "score2": 900, "endsAt": "2026-08-27T18:05:00.000Z" }

// room_event – seat_changed
{ "type": "room_event", "event": "seat_changed", "roomId": "clr123",
  "userId": "u1", "seatIndex": 3, "previousSeatIndex": -1, "ts": 1756318203912 }

// room_event – mic_changed
{ "type": "room_event", "event": "mic_changed", "roomId": "clr123",
  "userId": "u1", "micOn": true, "ts": 1756318203912 }

// room_event – user_joined / user_left
{ "type": "room_event", "event": "user_joined", "roomId": "clr123",
  "userId": "u1", "name": "Ayşe", "image": "https://<cdn>/avatars/u1.jpg", "ts":
1756318203912 }

// room_event – owner_changed
{ "type": "room_event", "event": "owner_changed", "roomId": "clr123",
  "newOwnerId": "u2", "newOwnerName": "Mehmet", "ts": 1756318203912 }

// room_event – voice_request
{ "type": "room_event", "event": "voice_request", "roomId": "clr123",
  "userId": "u1", "userName": "Ayşe", "avatar": null, "requestId": "vr1",
  "message": "Söz istiyorum", "expiresAt": "2026-08-27T18:02:00.000Z", "ts":
1756318203912 }

// room_event – pk_invite
{ "type": "room_event", "event": "pk_invite", "roomId": "clr123",
  "battleId": "pk1", "battle": { /* tam PK nesnesi */ }, "ts": 1756318203912 }

// presence – 10 sn'de bir TAM liste (delta değil)
{ "type": "presence", "onlineCount": 12, "totalCount": 12, "users": [
  { "id": "u1", "name": "Ayşe", "nickname": "Ayşe", "image": null,
    "lastSeen": "2026-08-27T18:00:00.000Z", "seatIndex": 3, "micOn": true,
    "chatRole": "op", "roleSymbol": "★", "roleLevel": 3, "isAdmin": false }
]}

// typing
{ "type": "typing", "users": ["Ayşe", "Mehmet"] }

```

```
// notification (bildirim akışı)
{ "type": "notification", "id": "n1", "notifType": "new_follower",
  "message": "Ayşe seni takip etti", "deepLink": "canlifal://profile/ayse",
  "createdAt": "2026-08-27T18:00:00.000Z", "read": false }
```

Tüm gerçek zamanlı olaylarda **ts** (epoch ms) alanı bulunur veya SSE **id**: satırı ile birlikte gelir.

12. EVENT ORDERING

- Olaylar sunucu içi bellek veri yoluna **zaman damgası** (**ts** , **epoch ms**) ile yazılır ve **artan sırada** iletilir.
- SSE **id**: satırı, o turda gönderilen **en yeni olayın ts** değeridir.
- Sohbet mesajları tek **messages** çerçevesinde **toplu** gelebilir — dizideki sıra doğrudur.
- **presence** bir olay değil **anlık tam görüntüdür**; sırası önemli değildir, en son geleni uygulayın.
- Yoklama aralığı **2 sn**'dir. Yani en fazla ~2 sn gecikme normaldir; olayların anında gelmesine dayanan UI yazmayın.
- **Farklı akışlar arasında sıra garantisi YOKTUR.** Oda akışındaki bir hediye ile bildirim akışındaki karşılığı arasında sıralama varsayımı yapmayın.

İstemci kuralı: her akış için **lastEventId** (en büyük gördüğünüz **ts**) tutun. Gelen olayın **ts** değeri bundan **küçükse** olayı yok sayın — yeniden bağlanma tekrarlarını böyle eleyn.

13. RECONNECT

Bağlantı koptuğunda:

1. Yerel ``lastEventId`` değerini sakla (en son SSE ``id:`` satırı).
2. Üstel geri çekilme ile yeniden bağlan: 1s → 2s → 4s → 8s → 16s → 30s (üst sınır 30s).
Her denemeye ± 20 rastgele sapma (jitter) ekle.
3. Yeniden bağlanma isteğine ``Last-Event-ID: <lastEventId>`` başlığını ekle.
Alternatif: ``?lastEventId=<ts>`` sorgu parametresi (başlık gönderilemiyorsa).
4. 403/404 alırsan yeniden deneme — kalıcı hatadır.
5. 401 alırsan token yenile, sonra bir kez daha dene.

Sunucu tarafı garantiler:

- Bellek veri yolu **son 2 dakika / 200 olay** tutar.
- **Last-Event-ID** bu pencere içindeyse **kaçırılan olaylar tekrar oynatılır**.
- Pencere dışındaysa akış “şimdiden” başlar → bu durumda **resync** (bkz. §14) yapın.
- **15 sn’de bir heartbeat** (: heartbeat) gönderilir. **35 sn** boyunca hiçbir bayt gelmezse bağlantıyı ölü sayıp kapatın ve yeniden bağlanın.

14. RESYNC

Aşağıdaki durumlarda **tam durum yeniden yükleme** yapın:

- Kopukluk > 2 dakika (olay tamponu penceresi aşıldı).
- Uygulama arka plandan öne geldi.
- Ekranı ilk giriş.
- Ardışık 3 yeniden bağlanma denemesi başarısız oldu.

Ekran	Resync ucu
Sohbet/sesli oda	GET /api/v1/chat/rooms/{roomId}/state
Canlı yayın	GET /api/v1/video-streams/{streamId}
PK	GET /api/v1/pk/{matchId}
Bildirimler	GET /api/v1/notifications?limit=30
Cüzdan	GET /api/v1/wallet
Genel	GET /api/v1/bootstrap

Resync sırası: **önce durumu çek** → **sonra SSE'yi aç** (lastEventId olmadan). Ters yapılsa durum ile olaylar arasında yarış oluşur.

15. DEEP LINKS

URI şeması: **canlifal://<type>/<value>**

type	Değer	Web karşılığı	Varlık
home	—	/	—
teller	id	/canli-falcilar/{id}	LiveFortuneTeller
room	id	/canli-oda/{id}	LiveSession
stream	id	/videolar/izle/{id}	VideoStream
video	id	/tiktok/{id}	ShortVideo
post	id	/fal/{id}	FortunePost
blog	slug	/blog/{slug}	BlogPost
profile	username	/profil/{username}	User
chatroom	slug	/sohbet/{slug}	ChatRoom
dream	slug	/ruya/{slug}	DreamInterpretation
dreamdict	slug	/ruya-sozlugu/{slug}	DreamDictionary
page	slug	/sayfa/{slug}	CustomPage
message	userId	/mesajlar/{userId}	User
question	slug	/sorbak/soru/{slug}	Question

Web yollarında /tr/ veya /en/ dil öneki opsiyoneldir ve çözümlemede tolere edilir.

Çözümleme ucu

```
GET /api/v1/deeplink/resolve?url=canlifal://teller/clx123
GET /api/v1/deeplink/resolve?url=https://canlifal.com/tr/canli-falcilar/clx123
```

Yanıt hedef varlığın var olup olmadığını ve tipini bildirir. Kullanım:

1. Bağlantıyı yerel olarak ayrıştır (hızlı yol).
2. Ekranı aç, iskelet göster.
3. resolve ucunu çağır — varlık yoksa “bulunamadı” ekranı göster.

Bildirimlerde gelen deepLink alanı **her zaman bu şemadadır** (bkz. §27).

16. FEATURE FLAGS

GET /api/v1/config?platform=ios # veya android | web | mobile | (boş = all)
 GET /api/v1/bootstrap?platform=ios # flags + config + kullanıcı özeti tek çağrıda

⚠️ **iki uç farklı şekil döner.** Bu bilinçlidir ve geriye dönük uyum için korunur.

GET /api/v1/config — hazır tüketim için (harita)

```
{
  "success": true,
  "data": {
    "featureFlags": { "AGENCY_ENABLED": true, "GIFTS_ENABLED": true },
    "featureFlagsDetailed": [
      { "key": "AGENCY_ENABLED", "enabled": true, "percentage": 100, "metadata":
null }
    ],
    "remoteConfig": {
      "gifts": { "gift_combo_windows": { "1x": 0, "10x": 5000 } },
      "leaderboard": { "leaderboard_periods": ["hourly", "daily", "weekly", "monthly", "seasonal"] }
    }
  },
  "request_id": "req_..."
}
```

GET /api/v1/bootstrap — ham liste + platform ayarları + kullanıcı özeti

```
{
  "success": true,
  "data": {
    "featureFlags": [ { "key": "AGENCY_ENABLED", "enabled": true, "percentage": 100,
"metadata": null } ],
    "remoteConfigs": [ { "key": "gift_combo_windows", "value": {"1x":0}, "valueType":
"json", "group": "gifts" } ],
    "platformSettings": {
      "credits_per_minute": "10", "jeton_tl_rate": "0.50",
      "vr_gift_receiver_percent": "70", "vr_room_owner_percent": "30",
      "vr_site_commission_percent": "50"
    },
    "user": null
  },
  "request_id": "req_..."
}
```

bootstrap kimlikli çağrıldığında user alanı profil özeti + okunmamış bildirim sayısı içerir. **Flutter için önerilen açılış çağrısı bootstrap 'tır** — tek turda her şeyi getirir.

platformSettings değerleri **string** olarak döner ("10" , "0.50"). İstemcide num.parse ile çevirin.

Kurallar:

- config.featureFlags[key] = enabled && percentage >= 100 (sunucu hesaplar).
- Kademeli yayın (%<100) için featureFlagsDetailed / bootstrap.featureFlags içindeki percentage ile kendi kullanıcı kovanızı hesaplayın.
- **Bilinmeyen bayrak = kapalı** varsayın. Eksik anahtar hata değildir.

- Yanıt **60 sn** sunucu tarafında önbelleklenir. İstemci en fazla 5 dakikada bir çeksın.
- Açılıştı bir kez + ön plana dönüşte bir kez yeterlidir.
- Bayrak durumu **UI'yi gizlemek/göstermek** içindir; **güvenlik sınırı değildir** — sunucu her isteęi ayrıca yetkilendirir.

17. REMOTE CONFIG

İki şekilde sunulur:

- GET /api/v1/config → remoteConfig : **gruba göre iç içe harita** ({ grup: { anahtar: deęer } }) — deęerler zaten valueType 'a göre çözümlenmiştir.
- GET /api/v1/bootstrap → remoteConfigs : **ham dizi** ({ key, value, valueType, group }), valueType ∈ string | number | boolean | json .

Gerçek anahtar örnekleri (canlı sistemden doğrulandı):

Anahtar	Grup	Tip	Amaç
gift_combo_windows	gifts	json	Hediye kombo pencereleri (ms) — { "1x":0, "10x":5000, "50x":3000, "100x":2000 }
leaderboard_periods	leaderboard	json	Liderlik tablosu periyotları
level_thresholds	levels	json	Seviye eşikleri — { "maxLevel":100, "xpPerLevel":1000 }
rtc_quality_thresholds	rtc	json	WebRTC kalite skoru eşikleri

Kurallar:

- **Her anahtar için istemcide varsayılan deęer bulundurun.** Sunucu deęeri yoksa/bozuksa varsayılanı düşün, hata vermeyin.
- valueType 'a göre ayırıştırın; json tipinde ayırıştırma hatası varsayılanı düşün.
- Bilinmeyen anahtarları yok sayın.

18. IMAGE / CDN RULES

- Tüm görsel/video URL'leri **mutlak HTTPS** adreslerdir. İstemci base URL eklemes.
- Kaynak: S3 uyumlu nesne deposu. Genel dosyalar doğrudan erişilebilir, özel dosyalar **imzalı URL** ile gelir.

- **İmzalı URL'ler süre sınırlıdır ve ASLA yerel olarak kalıcı saklanmamalıdır.** Süre dolmuşsa ilgili kaynağı yeniden çekin.
- Alan adı `null` veya boş string dönebilir → istemci yer tutucu göstermelidir.
- Önbellekleme: URL değişmedikçe içerik değişmez (dosyalar üzerine yazılmaz, yeni ad alır). `cached_network_image` ile güvenli önbelleğe alın.
- Avatar/rozet/hediye görselleri için disk önbelleği önerilir; hediye animasyonları (webp/mp4) ilk kullanımda ön-yükleyin.
- `Image.network` yerine hata geri dönüşü olan bir sarmalayıcı kullanın; 403 (imza süresi dolmuş) durumunda yer tutucu gösterin.

19. PAGINATION

İki mod desteklenir. **Varsayılan davranış değişmemiştir** (geriye dönük uyum).

A) Ofset modu (varsayılan)

```
GET /api/v1/notifications?page=2&limit=30
```

B) İmleç modu (opt-in, mobil için önerilen)

İmleç modu şu iki koşuldan biriyle etkinleşir:

- `?cursor=<id>` parametresi gönderildiğinde, **veya**
- `?paginate=cursor` parametresi gönderildiğinde (ilk sayfa).

```
GET /api/v1/notifications?paginate=cursor&limit=30 # ilk sayfa
GET /api/v1/notifications?cursor=clx9abc&limit=30 # sonraki sayfa
```

Yanıt meta :

```
{ "cursor": "clx9abc...", "hasMore": true, "limit": 30, "total": 7 }
```

`hasMore=false` → son sayfa. `cursor` bir sonraki isteğe aynen geçerilir.

İmleç destekleyen uçlar (12)

`/api/v1/notifications` , `/api/v1/user/activity` , `/api/v1/user/followers` , `/api/v1/user/following` , `/api/v1/user/received-gifts` , `/api/v1/user/likers` , `/api/v1/user/broadcast-history` , `/api/v1/messages` , `/api/v1/messages/{userId}` , `/api/v1/gifts/insights/me/history` , `/api/v1/admin/risk-events` , `/api/v1/agency/members`

`/api/v1/admin/audit-logs` farklıdır: zaman damgalı imleç kullanır (`?cursor=<ISO tarih>` → `nextCursor`).

Flutter kuralı: sonsuz kaydırmada **her zaman imleç modunu** kullanın. Ofset modu kayan listelerde tekrar/atlama üretir.

20. RATE LIMITS

Kapsam başına **dakikada** izin verilen istek sayısı:

Kapsam	Limit/dk	Uygulandığı yer
api_default	60	Genel
auth	10	Giriş/kayıt/parola
gift_send	10	Hediye gönderimi
lucky_gift	10	Şanslı hediye
chat_message	30	Oda mesajı
comment	20	Yorum
content_create	10	İçerik oluşturma
withdrawal	5	Para çekme
payment	10	Ödeme
stream_create	5	Yayın açma
room_create	5	Oda açma
pk_create	10	PK başlatma
agency_action	5	Ajans işlemleri
tip	10	Bahşiş
membership	5	Üyelik
report	5	Şikayet
upload	5	Dosya yükleme
rtc_telemetry	30	WebRTC telemetri

429 yanıtında dönen başlıklar:

```

Retry-After: 27
X-RateLimit-Limit: 10
X-RateLimit-Remaining: 0
X-RateLimit-Reset: 1756318230

```

Flutter kuralı: 429 alındığında `Retry-After` saniyesi kadar **bekleyin**; sıkı döngüde yeniden deneyin. Hediye/mesaj gibi kullanıcı tetikli işlemlerde butonu bu süre boyunca devre dışı bırakın.

21. UPLOAD RULES

Yükleme **iki adımlıdır**. Dosya asla API sunucusundan geçmez.

1. POST /api/v1/upload/presigned
 { "fileName": "foto.jpg", "contentType": "image/jpeg", "isPublic": false, "folder": "avatar" }
 → { "uploadUrl": "https://<imzali-url>", "cloud_storage_path": "...", "publicUrl": "https://<genel-url>" | null }
2. PUT <uploadUrl> (gövde = ham dosya baytları)
 Content-Type: image/jpeg ← 1. adımdaki contentType ile AYNI olmalı
3. cloud_storage_path değerini ilgili iş ucuna gönder
 (ör. profil güncelleme ucuna "image": "<cloud_storage_path>")

Kurallar:

- İzin verilen tipler: **image/*** **ve** **video/***. Diğerleri 400 döner.
- **folder** alanı sabit bir beyaz listeye eşlenir; bilinmeyen değer varsayılan klasöre düşer.
- PUT isteğinde **Content-Type birebir aynı olmalıdır**, yoksa depo 403 döner.
- İmzalı URL'de X-Amz-SignedHeaders içinde listelenen **her başlık** PUT isteğinde gönderilmelidir.
- Yükleme hız sınırı: **5/dk** (upload kapsamı).
- Kısa video için özel akış: POST /api/v1/short-videos/upload-url → PUT → POST /api/v1/short-videos/register.
- Yerel dosya yolları **asla** sunucuya gönderilmez; yalnızca **cloud_storage_path**.

22. ROOM BOOTSTRAP

Sesli/sohbet odasına giriş sırası:

1. GET /api/v1/chat/rooms/{roomId}/state
 → oda bilgisi, sahibi, aktif kullanıcılar, koltuklar, müzik/DJ durumu, aktif PK
2. POST /api/v1/chat/rooms/{roomId}/presence ← varlık bildir (periyodik tekrar)
3. GET /api/v1/chat/rooms/{roomId}/stream ← SSE aç (Authorization ile)
4. GET /api/v1/chat/rooms/{roomId}/messages?limit=50 ← geçmiş mesajlar
5. (sesli katılım için) POST /api/v1/chat/rooms/{roomId}/seats { "seatIndex": 3 }
 → 409 SEAT_TAKEN gelebilir; koltuk atama sunucuda işlem (transaction) içinde yapılır, yarış durumu yoktur. 409 alırsan state'i tazele.
6. (RTC için) GET /api/v1/trtc/usersig → ses/görüntü katmanı kimliği

Çıkışta: koltuktan kalk (DELETE .../seats), SSE'yi kapat, varlık yoklamasını durdur.

Varlık (presence) yoklaması: 60 sn'de bir POST .../presence . 300 sn güncellenmezse kullanıcı çevrimdışı sayılır.

23. LIVE BOOTSTRAP

Canlı yayın izleme sırası:

1. GET /api/v1/video-streams/{streamId} ← yayın meta verisi, yayıncı, izleyici sayısı
2. GET /api/v1/video-streams/{streamId}/stream ← SSE aç
3. GET /api/v1/trtc/usersig ← RTC kimliği (izleyici rolü)
4. RTC katmanına bağlan, video/ses akışını al
5. POST /api/v1/rtc/telemetry ← periyodik kalite ölçümü (aşağı bak)

Yayın açma (yayıncı):

1. POST /api/v1/video-streams ← yayın oluştur (rate limit 5/dk)
2. GET /api/v1/trtc/usersig ← yayıncı rolü
3. RTC'ye bağlan ve yayına başla
4. DELETE/PATCH /api/v1/video-streams/{id} ← yayını kapat

RTC telemetri

```
POST /api/v1/rtc/telemetry
{ "samples": [ { "context": "stream", "contextId": "vs1",
"rttMs": 120, "packetLossPct": 0.4, "jitterMs": 18,
"reconnectCount": 0, "freezeMs": 0 } ] }
```

- Toplu gönderim: en fazla **20 örnek** / istek. Hız sınırı **30/dk**.
- Önerilen aralık: **30 sn**'de bir tek örnek, veya 5 örneği 2,5 dk'da bir toplu gönder.
- Sunucu 0-100 kalite skoru ve seviye (excellent / good / fair / poor / critical) hesaplar.
- Eksik metrikler skorlamadan çıkarılır — gönderemediğiniz alanı **atlayın**, 0 göndermeyin.

24. PK FLOW

PK (Player Knockout) üç bağlamda çalışır: sohbet odası, canlı yayın, canlı fal odası.

DAVET → POST /api/v1/chat/rooms/{roomId}/pk { "action": "invite", "targetRoomId": "..."}
 (Idempotency-Key ZORUNLU önerilir – kapsam: pk_action)
 → karşı odaya SSE: room_event / pk_invite

KABUL → POST /api/v1/chat/rooms/{roomId}/pk { "action": "accept", "battleId": "..."}
 → her iki odaya SSE: type=pk, status=active

SKOR → PK sırasında gönderilen her hediye skoru artırır.
 Her skor değişiminde her iki odaya SSE: type=pk (score1, score2, endsAt)

BİTİŞ → Süre dolduğunda sunucu otomatik sonlandırır.
 SSE: type=pk, status=ended, winnerId

AKIŞ → GET /api/v1/pk/{matchId}/stream ← maça özel SSE akışı
 DURUM → GET /api/v1/pk/{matchId} ← resync
 AKTİF → GET /api/v1/pk/active ← devam eden maçlar
 BENİM → GET /api/v1/pk/me
 SİRALAMA → GET /api/v1/pk/leaderboard

Yayın PK'sı: POST /api/v1/video-streams/pk (kapsam stream_pk).

Canlı fal PK'sı: POST /api/v1/live/pk (kapsam live_pk).

İstemci kuralı: PK skorunu **asla yerel olarak hesaplamayın**. Sunucudan gelen score1 / score2 tek gerçektir; yerel iyimser güncelleme yapılacaksa SSE geldiğinde üzerine yazın.

endsAt sunucu saatidir. Geri sayımı endsAt - sunucu_saati_farkı ile hesaplayın; cihaz saatine körü körüne güvenmeyin.

25. GIFT FLOW

1. KATALOG GET /api/v1/gifts/catalog (sürüm kontrolü: /api/v1/gifts/version)
 2. BAKİYE GET /api/v1/wallet
 3. GÖNDER POST /api/v1/gifts/send ← Idempotency-Key ZORUNLU
 veya POST /api/v1/chat/rooms/{roomId}/gifts (oda içi)
 veya POST /api/v1/video-streams/{streamId}/gifts (yayın içi)
 4. SONUÇ → yanıt: yeni bakiye + hediye kaydı
 → odadaki/yayındaki herkese SSE: type=gift
 5. ANİMASYON → SSE olayındaki giftImage/quantity ile oynatılır

Idempotency (ZORUNLU)

Tüm finansal uçlarda Idempotency-Key başlığı gönderin:

Idempotency-Key: <UUID v4>

Uç	Kapsam
POST /api/v1/gifts/send	gift_send
POST /api/v1/chat/rooms/{roomId}/gifts	chatroom_gift
POST /api/v1/video-streams/{streamId}/gifts	stream_gift
POST /api/v1/chat/rooms/{roomId}/pk	pk_action
POST /api/v1/video-streams/pk	stream_pk
POST /api/v1/live/pk	live_pk
POST /api/v1/gifts/missions/{missionId}/claim	mission_claim

Davranış:

- Aynı anahtarla ikinci istek → **409 DUPLICATE_REQUEST** (yeniden işlem YAPILMAZ).
- Anahtar **24 saat** saklanır.
- Anahtar **istek başına yeni üretilir**, yeniden deneme sırasında **aynı** anahtar kullanılır.
- Ağ zaman aşımında güvenli yeniden deneyebilirsiniz — çifte harcama olmaz.

Diğer hediye uçları

Uç	Amaç
GET /api/v1/gifts/types	Hediye kategorileri
GET /api/v1/gifts/recent-big	Son büyük hediyeler
POST /api/v1/gifts/lucky	Şanslı hediye (rate limit 10/dk)
GET/POST /api/v1/gifts/battles	Hediye savaşları
GET /api/v1/gifts/goals	Hediye hedefleri
GET /api/v1/gifts/missions	Görevler
GET /api/v1/gifts/insights/me/history	Kendi hediye geçmişim (imleç destekli)
GET /api/v1/gifts/check-reciprocal	Karşılıklı hediye kontrolü

26. WALLET FLOW

BAKİYE	GET	/api/v1/wallet	→ { coins, jetonBalance, cfcBalance, credits, ... }
JETON	GET	/api/v1/jeton	→ jeton hareketleri / günlük kazanç
PAKETLER	GET	/api/v1/credit-packages	→ satın alınabilir jeton paketleri
SATIN AL	POST	/api/v1/payments	→ ödeme başlat (rate limit 10/dk)
ÇEKİM	POST	/api/v1/withdrawals	→ para çekme talebi (rate limit 5/dk)
GÜNLÜK	POST	/api/v1/daily-login	→ günlük giriş ödülü
GÖREVLER	GET/POST	/api/v1/daily-missions	

Kurallar:

- **Bakiye tek gerçek kaynağı sunucudur.** Hediye gönderdikten sonra yanıt içindeki yeni bakiyeyi uygulayın; yerel çıkarma yapmayın.
- Yetersiz bakiye → **422 INSUFFICIENT_BALANCE**, details içinde required / available .
- Günlük giriş ve görev ödülleri sunucuda işlem (transaction) + tekrar koruması içinde verilir; çift tıklama çift ödül vermez.
- Para çekimi KYC gerektirebilir → **403 KYC_REQUIRED** .
- Ödeme akışı harici sağlayıcıya yönlendirir; dönüşte cüzdanı **yeniden çekin** (webhook gecikmesi olabilir, ~birkaç saniye).

27. NOTIFICATION FLOW

İki kanal

Kanal	Ne zaman
Push (OneSignal)	Uygulama kapalı/arka planda
SSE /api/v1/notifications/stream	Uygulama açık ve ön planda

Uçlar

GET	/api/v1/notifications?paginate=cursor&limit=30	# liste (imleç destekli)
GET	/api/v1/notifications/stream	# canlı akış
POST	/api/v1/notifications	# okundu işaretle
	{ "notificationIds": ["n1","n2"] } veya { "markAll": true }	
DELETE	/api/v1/notifications	# bildirim(leri) sil
POST	/api/v1/devices/fcm	# push token kaydı
DELETE	/api/v1/devices/fcm	# push token kaydını sil

Bildirim nesnesi

```
{
  "id": "n1",
  "type": "new_follower",
  "message": "Ayşe seni takip etti",
  "deepLink": "canlifal://profile/ayse",
  "read": false,
  "createdAt": "2026-08-27T18:00:00.000Z",
  "fromUser": { "id": "u1", "name": "Ayşe", "image": null }
}
```

Tekilleştirme (sunucu tarafı)

Sunucu aynı bildirimi belirli bir pencere içinde **tekrar üretmez** — ne kayıt ne push:

Tür	Pencere
Varsayılan	60 sn
follow / new_follower	6 saat
like / post_like / comment_like	1 saat
achievement / level_up	24 saat
live_started / stream_started	30 dk

Yani 3 art arda mesaj → 3 mesaj teslim, **1 bildirim**. İstemcide ek tekilleştirme gerekmez.

Derin bağlantı

deepLink alanı her zaman canlifal://... şemasındadır (bkz. §15). Eski kayıtlarda alan null olabilir — okuma anında sunucu türetmeye çalışır, yine de null gelirse bildirime tıklandığında **bildirim listesinde kalın**, çökmeyin.

Push token yaşam döngüsü

```
Giriş sonrası      → POST /api/v1/devices/fcm { token, platform }
Token yenilenince  → POST /api/v1/devices/fcm (yeni token)
Çıkışta           → DELETE /api/v1/devices/fcm + POST /api/v1/auth/logout
```

28. DEVICE / SESSION FLOW

Cihaz kimliği

- Her kurulum kalıcı bir cihaz kimliği üretir ve güvenli depoda saklar.
- Cihaz kimliği kimlik uçlarına gönderilir; çoklu hesap tespiti ve risk skorlaması için kullanılır.

Cihaz doğrulama

```
POST /api/v1/auth/verify-device # yeni cihazı doğrula
POST /api/v1/auth/reclaim-device # cihazı başka bir hesaba taşı
```

Yeni bir cihazdan giriş yapıldığında sunucu doğrulama isteyebilir. Bu durumda giriş yanıtı token yerine doğrulama gerektiğini belirtir.

Oturum yaşam döngüsü

```
AÇILIŞ
├─ Güvenli depodan tokenları oku
├─ GET /api/v1/bootstrap?platform=<ios|android>
│   └─ featureFlags + remoteConfigs + kullanıcı özeti + okunmamış bildirim sayısı
├─ Token yoksa/geçersizse → giriş ekranı
└─ POST /api/v1/devices/fcm (push token)

ÖN PLANA DÖNÜŞ
├─ GET /api/v1/bootstrap (yenile)
├─ Aktif ekranın resync ucunu çağır (§14)
└─ SSE akışlarını yeniden aç

ARKA PLANA GEÇİŞ
├─ SSE akışlarını kapat (pil tasarrufu)
└─ Varlık (presence) yoklamasını durdur

ÇIKIŞ
├─ POST /api/v1/auth/logout
├─ Push token kaydını sil
├─ Güvenli depoyu temizle
└─ Tüm SSE akışlarını kapat
```

Sağlık kontrolü

```
GET /api/v1/health
→ { "success": true, "data": { "status": "ok", "uptime": 376.5, "dbLatencyMs": 17,
"timestamp": "..." } }
```

503 dönerse servis geçici olarak kullanılamıyordur → bakım ekranı gösterin, üstel geri çekilme ile tekrar deneyin.

EK — FLUTTER İSTEMCİ KONTROL LİSTESİ

- [] Tek HTTP istemcisi (Dio/http) + interceptor: Authorization , Accept , request_id loglama
- [] 401 → tek seferlik token yenileme (mutex korumalı)
- [] 429 → Retry-After kadar bekle, buton kilidi
- [] Hem zarflı hem legacy yanıtı anlayan tek ayrıştırıcı
- [] error.code üzerinden dallanma, message yalnızca gösterim
- [] Finansal POST'larda Idempotency-Key (UUID v4, yeniden denemede aynı)
- [] SSE: Last-Event-ID , 35 sn heartbeat zaman aşımı, üstel geri çekilme + jitter
- [] ts tabanlı olay tekilleştirme

- [] Bilinmeyen `type / event / bayrak/config` anahtarı → sessizce yok say
- [] Sonsuz kaydırmada imleç modu (`?paginate=cursor`)
- [] Arka plana geçişte SSE kapatma
- [] Her remote config anahtarı için yerel varsayılan
- [] Tokenlar yalnızca `flutter_secure_storage` 'da

SÜRÜM GEÇMİŞİ

Sürüm	Tarih	Değişiklik
1.0	2026-08-27	İlk sürüm — 28 başlık eksiksiz (Faz 24)